

Programming

Objectives

After this course, the student:

- can write simple Java programs from scratch
- knows the principles of object-oriented programming
- can, with aid, design simple object-oriented programs
- can use the Swing framework
- knows some principles of code quality and can apply these

Content

- Elementary imperative language concepts: variable, assignment, if-else, while, method, parameter, function, types, arrays, scope, I/O.
- Some advanced topics: some collection classes, recursion.
- Object-oriented design and programming concepts: object, class, instance variable, instance method, inheritance, overriding, polymorphism, interface.
- GUI: Swing, event handling, simple graphics, GUI components (window, button), simple animation.

Logic and set theory

Objectives

After completing the course, the student can work with formulas from propositional logic and predicate logic, and knows their meaning and

use. In particular, the student can 'reason' with logical formulas in a system with assumptions, deductions and conclusions. The student can put his logical knowledge to work, in particular to prove (or refute by means of a counterexample) formulas of set theory, having developed both the

skills and the understanding of the fundamental mathematical abstractions in the area of sets, including relations, functions, orderings and induction.

Content

Subjects: Propositions, truth tables, equivalence, logical consequence, tautology, contradiction, contingency. Predicates, quantifiers, variable binding. Logical derivation, reasoning with propositions and predicates, conclusion, assumption, context, validity.

Set, subset, intersection and union, complement, difference, the empty set, powerset, cartesian product.

Relation, equivalence relation, class, partition.

Mapping (function), image and source, injection, surjection, bijection, inverse function, composition of relations and functions.

Partial ordering, linear ordering, Hasse diagram, maximal and minimal elements.

Induction, strong induction, inductive definition.

Calculus

- Algebraic Skills, Functions (C1)
Recapitulation of elementary algebra and high school mathematics with emphasis on algebraic skills (such solving inequalities), Cartesian coordinates, and functions; in particular, algebraic manipulation of trigonometric, exponential and logarithmic functions.
- Limits (C2)
Understanding the conceptual definition of limits; able to compute various types of limits; able to determine whether a function is continuous, or can be extended to a continuous function.
- Differentiation I (C3a)
Understanding the notion of differentiable functions; able to interpret derivatives in terms of tangent line; able to compute derivatives using the product, quotient, and chain rules; able to use implicit differentiation.

- Differentiation II (C3b)
Higher order derivatives; linear approximations of a function; Taylor polynomials; l'Hopital's rule and Taylor polynomials as tools to determine limits.
- Transcendental functions(C4)
Able to determine inverse functions of one-to-one (injective) functions; in particular, knowing the main properties of the natural logarithm (as inverse of the exponential function) and of the inverse trigonometric functions.
- Integration(C5)
Able to compute definite and improper integrals using various techniques such as integration by substitution, integration by parts, and partial fractions; understanding sums and Sigma notation, Riemann sums.
- First-Order Differential Equations(C6a)
Able to solve simple first-order differential equations (separable differential equations) and linear first-order differential equations.
- Vectors in the Plane and in Space(L1a)
Able to find equations and parametric (vector) equations for lines in the plane and in space, for planes in space;
able to interpret and compute with dot products and cross products;
able to determine lengths of vectors, distances and angles between vectors.

Content

- Algebraic Skills, Functions(C1)
Recapitulation of elementary algebra and high school mathematics with emphasis on algebraic skills (such solving inequalities), Cartesian coordinates, and functions; in particular, algebraic manipulation of trigonometric, exponential and logarithmic functions.
- Limits(C2)
Conceptual definition of limits; various types of limits; concept of continuity.
- Differentiation I(C3a)
Differentiable functions; interpretation of derivatives in terms of tangent line; computation of derivatives using the product, quotient, and chain rules; implicit differentiation.

- Differentiation II (C3b)
Taylor polynomials of functions; l'Hopital's rule for computing limits.
- Transcendental functions(C4)
Inverse functions of one-to-one (injective) functions; in particular, the main properties of the natural logarithm (as inverse of the exponential function) and of the inverse trigonometric functions.
- Integration(C5)
Computation of definite and improper integrals using various techniques such as integration by substitution, integration by parts, and partial fractions; sums and Sigma notation, Riemann sums.
- First-Order Differential Equations(C6a)
Simple first-order differential equations (separable differential equations) and linear first-order differential equations.
- Vectors in the Plane and in Space(L1a)
Equations and parametric (vector) equations for lines in the plane and in space, for planes in space; dot products and cross products; lengths of vectors, distances and angles between vectors.

Linear Algebra

Objectives

- Being able to calculate with matrices and vectors, including various norms, angles between vectors.
- Understanding their role in applications such as recommender systems.
- Being able to solve linear systems with Gauss or Gauss-Jordan elimination.
- Understanding their role in applications such as splines (for instance for Computer Graphics).
- Being able to solve linear least squares problems, including those coming from fitting.
- Being able to compute determinants of small matrices, and knowing several useful applications.

- Being able to compute the eigenvalue decomposition and singular value decomposition for small matrices.
- Understanding the role of eigenvalues and eigenvectors in Google PageRank and other Markov chain applications.
- Understanding the basics of eigenvalue based clustering techniques.
- Understanding the role of singular values and singular vectors in web analytics, data mining, and text mining.
- Understanding the linear algebra notions of rank, low rank, orthogonality, (in)dependency, positive definiteness.
- Being able to implement linear algebra methods in Python, Julia, Matlab or another language of choice.
- Being able to compute reflections and projections in 2D.
- Understanding the role of finite precision arithmetic on computers.
- Being able to do computations relating to the math of finite precision arithmetic.

Content

Informal description

This course offers a wide range of interesting linear algebra techniques and very nice applications, including Big Data applications. These techniques are almost indispensable for the current Big Data era, and are also relevant for the field of Artificial Intelligence, including Deep Learning.

You will learn how to solve linear systems, arguably the most common and important scientific problem.

Least squares methods (or regression, very useful in Engineering) can be used to determine an approximate line (or polynomial, or spline) through a number of points (used in fitting problems, Computer Graphics, games, movies).

Rotations and reflections may be useful for Computer Graphics and Robotics.

Angles between vectors can be used to compare tastes of movies, music, or books, and to predict such tastes for the future.

You will learn the idea behind Google PageRank and its connection with eigenvalues, how it is computed, and you can also apply similar techniques to finally determine which team or country is the best soccer team.

We can use text mining methods to determine the most important tweets or keywords from a set of tweets or similar data from social media; useful for effective communication. Similar data mining techniques may be used to study customer behavior.

The course is designed to be interesting for many students, including (but not restricted to!) students from Computer Science, Data Science, P&T, and Robotics.

The subjects are fascinating on their own, but also form an ideal preparation for subsequent courses such as Data Mining, Machine Learning, Numerical Linear Algebra, Computer Graphics, and Optimization.

The course offers a nice combination of theoretical (math) and practical (implementation) work, partly done in groups.

You can use a language you might already know (for instance Python, Matlab, R) or learn a new language (for instance Julia).

Content

Matrices, vectors

- Operations, matrix and vector norms, angles between vectors
- Applications in recommender systems
- Linear systems and least squares problems
- Gaussian elimination, Gauss and Gauss-Jordan form, solutions, rref, inverse matrices
- Normal equations
- Applications in data fitting
- Rotations, projections, and reflections
- Corresponding matrices
- Applications in Computer Graphics and robotics
- Eigenvalues and eigenvectors
- Eigenvalue decomposition
- Determinants
- Google PageRank, clustering, and other applications in Markov chains

- Singular value decomposition (SVD/PCA)
- Singular values and vectors
- Orthogonality, rank, linear dependency, low-rank matrices, TSVD
- Applications in Web Analytics and data/text mining, feature extraction
- Implementing linear algebra
- Flops and memory costs, finite precision arithmetic and rounding errors
- Linear algebra with Python, Julia, Matlab, or another language of choice

Foundations of Data Analytics

Objectives

General learning goals

- Use basic statistical concepts and techniques and perform appropriate statistical tests
- Choose and apply suitable visualization techniques
- Analyze and model data using linear regression, clustering, decision tree mining and association rules learning
- Read and make simple database schemes and simple queries to a database.
- Clean data, choose and apply data transformations, data reduction, and data discretisation
- Understand, interpret, and document data and information in the context of realistic scenarios, structure open problems along the phases of the data life cycle, formulate hypotheses, incorporate ethical considerations and reflect on analysis strengths and limitations
- Use tools for implementing data engineering tasks (Python with Jupiter Notebooks) in a structured way
- Choose and communicate interesting findings in the language understandable for their end user (visually or textually).

Discrete Structures

Objectives

By the end of the course students

- 1) are familiar with and can apply basic proving techniques (e.g., proof by contradiction, induction, double counting) and can write correct and clear proofs with them.
- 2) know the basic definitions and concepts of graph theory and the important relations between them.
- 3) are familiar with important graph structures and graph problems (e.g., Euler tours, Hamiltonian Cycles, 2-connectivity).
- 4) have a thorough understanding of rooted trees (e.g., properties like height, depth).
- 5) know the basics of combinatorics and counting (e.g., number of binary strings of given length, number of subsets of a set, number of permutations of a set, inclusion exclusion counting).

Content

In this course we will cover basic discrete mathematical structures that occur in many contexts in computer science. You will learn the basic skills and knowledge to formally reason about these structures. Among others we will cover relations and orderings, counting arguments, and graphs and trees.

Probability and Statistics for Computer Science

Objectives

Students are able to

- Perform elementary calculations using the rules of probability theory
- Work with discrete and continuous random variables and apply the important standard distributions in appropriate settings

- Use joint distributions and properties when using multiple random variables such as independence and the Central Limit Theorem
- Use discrete Markov chains and determine their properties and limiting distributions
- Perform an exploratory data analysis of a data set
- Identify properties of point estimators
- Construct and interpret confidence intervals in various relevant settings
- Apply hypothesis testing in various relevant settings and interpret the results
- Explain how probability and statistics can be used in practice, for example in the analysis of random algorithms and real-world data sets

Content

This course gives an introduction to probability and statistics. Uncertainty shows up in many places within computer science. In complex systems you may need to consider failure rates of components, communication delays or imprecision in data or sensor input, whereas in algorithms randomness is used as a tool to create simpler or faster solutions. In these situations considering only best and worst-case behavior leads to an overly optimistic or pessimistic representation. Instead, a better option is often to model this uncertainty and analyze the model. Probability theory gives us the tools to do this and get a more accurate representation of the behavior of complex systems or algorithms. Statistics, on the other hand, concerns what can be learned from data. In this course you will learn how to build probabilistic models, perform statistical analyses of real data, and the theory behind all this.

- Sample space, events, interpretations of probability, axioms of probability
- Probability rules, conditional probability, independence, Bayes' rule, random variables
- Discrete random variables, probability and cumulative distribution functions, expectation, variance
- Standard discrete distributions such as the uniform, binomial and Poisson distribution

- Continuous random variables, probability density function, cumulative distribution function, expectation, variance
 - Standard continuous random variables such as the uniform, normal and exponential distribution, Poisson process
 - Central Limit Theorem (CLT) and its applications
 - Discrete joint distributions, marginal and conditional probability, independence, correlation
 - Linear combination of (normal) random variables
 - Discrete Markov chains and their properties, limiting distribution
 - Exploratory data analysis, population and sample mean, variance, standard deviations, Box plot and probability plot
-
- Sample distribution and the CLT
 - Point estimators and their properties, bias, mean-squared error, relative efficiency
 - Confidence intervals for parameters of the normal distribution and proportions
 - Hypothesis testing, Type I and Type II errors, power of a test, p -value
 - Tests for the parameters of the normal distribution and proportions
 - Connection between confidence intervals and hypothesis testing
 - Examples of using probability and statistics to analyze the performance of random algorithms and real-world data sets

Data Structures

Objectives

There are many aspects to the study of data structures, and the algorithms that operate upon them. In this course, the student will learn the basic skills and knowledge to develop efficient algorithms to solve computational problems and to make informed choices between different solutions for the same problem. These include standard data structures and algorithms for frequently appearing

problems, algorithm design techniques, how to establish that an algorithm is correct, and how to analyse the efficiency of an algorithm.

Content

Design techniques: Incremental algorithms, recursion, divide & conquer. Correctness: induction and invariants. Efficiency analysis: O-notation, recurrences. Sorting: MergeSort, InsertionSort, HeapSort, sorting in linear time, lower bounds for sorting. Selection Algorithms. Data structures: abstract data structures, heaps, hash tables, search trees (incl. red-black trees), augmenting data structures, union-find. Basic graph algorithms: adjacency list, adjacency matrix, DFS, BFS, topological sort, minimum spanning trees.

Computer Systems

Objectives

- Knowledge: At the end of the course, students understand how both combinatorial and sequential digital circuits can be constructed using elementary logical gates. They understand how a data path of a simple processor with registers, busses and an ALU can be constructed and how assembly instructions can be implemented on this structure. They can write simple assembly programs and know how higher order languages can be translated to assembly programs. They understand the embeddedness of a processor in a computer, including notions such as caches, multi-threading, networking, the operating system and virtual memory.
- Skills: At the end of the course students can perform the following tasks.
 - They can reason with and simplify propositional formulas, using truth tables, algebraic rewriting, and/or Karnaugh maps;
 - They can design simple combinatorial circuits;
 - They can calculate with positive and negative numbers in different number bases, especially in base 2.
 - They can construct state machines problem descriptions and are able to transform these into sequential circuits;

- They can implement assembly instructions onto simple processor layouts, including fetching the instruction from memory.
- They can construct assembly language programs for simple tasks using the PP2 instruction set and can translate algorithms in a Java like language into assembly

Content

Combinatorial circuits: switching patterns like invert, and, or; physical quantities, like voltages and currents, and discrete digital information, like bits; from relays and transistors to basic gates; truth tables, (algebraic) propositional calculus, Karnaugh maps, normal forms; designing two- and multi-level circuits.

Sequential circuits: the concept of state to represent the past; finite state machines; Mealy and Moore machines; feedback to implement states;

flip-flops: R-S, latch, clocked; designing sequential circuits; Number representations and conversions;

Arithmetic circuits: adders, subtracters, multipliers, shifters;

Computer building blocks: ALUs, registers, data paths, control units, memory, input & output, interrupts;

Instruction sets: Machine Language and Assembly Language; instruction formats; programming in Assembly Language; translating programming language constructs;

Aspects of Operating Systems: concurrency, synchronization, semaphores, communication, buffering, virtual memory.

Software Design

Objectives

1. Students will be able to compare major software development processes and to argue about which process to apply in different scenarios.
2. Students will be able to practice the use of version control systems (VCSs) by developing software solutions individually and as a small team.

3. Students will be able to translate requirements into specifications that peers can understand and use.
4. Students will be able to employ software design principles & design patterns while developing a software solution and to argue when these are feasible.

Ethics of Technology and Engineering

Objectives

The overall aim of Ethics of Technology and Engineering is to introduce students to the role of ethics in engineering and innovation. It covers the following learning outcomes

- Basic ability to reflect on ethical aspects of engineering in a societal context
- Basic ability to conduct a normative analysis of engineering with the help of ethical concepts (such as values, risks and (shared) responsibility)
- Basic ability to conceive and evaluate solutions to engineering challenges with the help of classical ethical theories (such as utilitarianism, deontology and virtue ethics)

Content

How does one engineer for a rapidly changing world? And how can one use engineering to make the world a better place? In order to be successful, technological innovations must take into account the societal and ethical aspects of technology. Future engineers will work in multidisciplinary teams – whether they work in high-tech startups, large established companies, government agencies, or research institutions. They will be expected to cooperate and combine their technological expertise with ethical skills in order to realize successful and responsible innovations that improve society while at the same time addressing ethical concerns about new technologies.

Ethics of Technology and Engineering introduces students to normative skills that help engineers, designers, and researchers to make decisions concerning ethical questions. These skills are needed e.g., to understand the complex societal context of sustainable mobility and energy systems, to deal with and to identify privacy aspects of all kinds of smart systems, to help solve controversial health and risk issues, etc. Students acquire insights into basic ethical concepts

(such as values, risks and responsibilities) and analyse the role these concepts play in the context of the development and design of technologies. Furthermore, students will get acquainted with codes of conduct (such as the TU/e code) and classical ethical theories (such as utilitarianism, deontology, and virtue-ethics) and learn how to apply these theories to ethical decisions making in the field of ethics of technology and engineering ethics.

Students will work in groups on a case related to their interest and field of study. They will analyze their case with the help of the "ethical cycle": a systematic step-by-step problem-solving method meant to identify relevant ethical issues and recommend ways to tackle them in the light of ethical theories. *Ethics of Technology and Engineering* will offer different themes that students can enroll in, tailored to the interests and to the fields of study of the different students of the TU/e.

CBL Embedded Systems

Challenge-based Learning (CBL)

Objectives

Keywords for this DBL are:

- model-based design and code generation for embedded systems
- interaction between software in a machine and its physical environment
- failure detection and recovery
- cooperation, project management, planning & organization, meeting techniques
- presentation, reporting, and self-reflection

Content

For each group of 6 participants:

- Support (e.g. training) for the professional skills Cooperating and Presenting in the SkillsLab
- General SCRUM-training, and a special SCRUM-master training for one of the group members
- Global specification and design of the machine to be constructed;
- Detailed specification and design of the machine;
- Building the machine using Fischer Technik, Raspberry Pi, and self-designed electronics.
- Programming the machine, optionally using code generation.

- Optionally connecting the machine over Ethernet to other machines on the same workfloor.
- Testing the machine, optionally testing the code first using simulation in the loop.
- Considering fault-detection and handling during the entire design phase.
- Documenting progress in a light-weight but effective way throughout the project using living documents.
- Presenting progress and demo-ing final results to other groups.

Software Specifications

Objectives

The student can describe existing software systems or software systems to be developed by means of informal and formal specification techniques. The student can reason about specifications, and can assess the quality of specifications. The student can use tools to create specifications and to verify that specifications meet requirements expressed as properties. The student also understands and can apply model-based testing techniques to relate software systems to their specifications

Content

In this course, we study a number of specification techniques and concepts pertaining to software systems. The course covers informal and formal techniques. The informal techniques include, among others, UML diagrams, for instance, use-case diagrams, class diagrams, sequence diagrams. The formal techniques include temporal logics and modelling formalisms like labelled transition systems. We address the quality of specifications. In the assignments, the knowledge of modelling and specification is put to practice. The course also covers foundations and applications of model-based testing techniques

Networks

Objectives

The learning objectives are:

- i) Understanding the organization of computer networks, including heterogeneous networks of networks such as the Internet, or the Internet of Things,

- ii) understanding the Internet structure and how standard problems are solved in that context,
- iii) being able to analyze simple network protocols, and
- iv) being able to independently study literature concerning computer networks.

Content

The OSI model; notions: protocol, connection, application, service; network topology, network components and physical media; overview of relevant services; the link layer; the network layer and routing; the Internet protocol; naming; addressing; the transport layer: TCP, UDP; reliability and multiplexing; performance characteristics and quality of service

Functional Programming

Objectives

The goal of this course is to acquaint students with the declarative programming paradigm, mainly through pure and lazy functional programming with leading roles for types, generalization, and calculation.

The course is to have theoretical and practical elements of which the latter will be using Haskell.

Content

Derivation and construction of functional programs, mainly by calculation.

Recursion, inductive types, recursion schemes, monads, streams and some theoretical elements of lambda calculus and type theory (polymorphic types).

Operating Systems

Objectives

At the end of the course, the students are able to:

- explain the most relevant concepts and techniques used in operating systems;

- identify problems with concurrent processes execution, atomicity and interference, and the effects they have on programs' correctness;
- implement simple synchronization and communication protocols between threads and processes;
- analyze synchronization and communication protocols between threads and processes to check whether they respect invariants and avoid deadlocks;
- apply memory and file system management techniques on example systems.

Through the course the student will also gain experience with programming in C.

Content

Introduction of concurrency, atomicity and interference. Program traces and interference. Reasoning about concurrent programs using traces, invariants and assertions. Standard problems like mutual exclusion, bounded buffer and readers/writers. Synchronization using semaphores, conditional critical sections and monitors. Hardware primitives. Global description of operating system tasks. Operating System architecture and Application Programmers interface. The kernel, multiprocessing, interrupts, device drivers, implementation aspects. Virtual memory: demand paging, replacement algorithms. Resource management, scheduling. Filesystems. Examples from Linux and Windows

Practical exercises in a virtual linux environment

Datamodeling and Databases

Objectives

This course teaches students to design (the data structures of) information systems. With "design" we mean that the students learn to translate a natural language description of an organization's information needs into a data model, expressed in the entity relationship model. Students must be capable of translating that model into a relational database structure. They must be able to optimize this structure by means of constraints and decomposition algorithms.

Students also must become fluent in query languages, in order to express natural language questions in query languages, and in order to express formal queries in natural language.

Students additionally acquire social awareness of the role of conceptual modeling in social and technical systems and can effectively apply this awareness in the design of conceptual models.

Content

- Entity-Relationship Model (and equivalent UML constructs)
- Designing E-R diagrams for business situations
- The Relational Database model
- Constraints and Decompositions
- Query languages: translation from natural language to
- query languages and back; equivalence of query languages.

Automata and Formal Languages

After completion of the course the student knows the concepts of a deterministic finite automaton (DFA) and of a non-deterministic finite automaton (NFA) with silent moves, is able to identify the languages accepted by these automata and is able to construct an automaton of either kind, minimal in the case of DFA, that accepts a given language. The student understands the equivalence of regular expressions, DFAs and NFAs.

The student knows the concepts of a push-down automaton (PDA) and of a context-free grammar (CFG), and is able to identify the languages accepted by a PDA and generated by a CFG. The student understands the notion of a parse tree and the notions of top-down and bottom-up parsing. The student understands the equivalence of PDAs and CFGs, a number of closure properties of the class of context-free languages.

The student is able to apply and prove the pumping lemma for the classes of regular languages and context-free languages.

The student understands the notion of a Turing machine (TM), of a decidable language and of a recognizable language. The student is able to prove that a specific language is or is not decidable or recognizable.

Content

This course covers fundamental concepts and theory in the fields of Automata Theory and Formal Language Theory including regular languages, context-free languages, and decidable languages and

their computational models, viz. NFAs, PDAs, and TMs. In addition NP-completeness is treated. The above is covered by Chapters 1 to 5 of the textbook for this course.

Provable Programming

Objectives

The course is part of *Foundations of program construction and system behavior*.

After this course, the student:

- can implement (parts of) given algorithms and prove them correct using Dafny
- can specify programs, and functions rigorously and formally
- can prove these programs correct with the aid of Dafny
- can reason about and understand programs in a modular, contract-based way
- can specify programs, modules, and functions rigorously and formally
- knows several tooling approaches for static verification, of which one in depth (Dafny)
- understands the theory of assertional program verification

knows the concept of program derivation (Dijkstra's Eindhoven school) and can apply this approach to simple cases

Introduction to Numerical Analysis

Objectives

By the end of the course, the student should be able to

- understand, explain, and recognize the effect of floating-point representation, machine precision, round-off errors, and condition number on the accuracy of numerical approximations
- use basic interpolation techniques to approximate functions

- approximate the integral of functions using basic numerical integration techniques
- approximate the derivatives of functions using difference formulas
- numerically solve for the solution of nonlinear algebraic equations
- analyse numerical methods with respect to accuracy (or error), stability, and convergence
- numerically solve (systems of) ordinary differential equations
- apply numerical methods to compute the solution to a realistic modeling problem using scientific computing software (e.g., Python), and evaluate the error in the obtained approximation

Content

Largely due to the rapid progress in both hardware and numerical algorithms, computer simulations of mathematical models have inarguably become indispensable in engineering, the physical sciences, as well as in life sciences, social sciences, medicine, business, and even the arts. Furthermore, there is evidence that our present ability to solve complex problems depends not only on faster hardware, but even more so on efficient algorithms.

This course introduces the student to the basic concepts of numerical analysis --- the study of the numerical methods and algorithms that are at the core of computer simulations, and which use numerical approximations to solve problems in mathematical analysis that cannot be solved exactly. The numerical methods will be analysed with respect to their properties (such as accuracy) and effectiveness, and will be applied to simple, real-world problems using scientific computing software.

- Finite precision arithmetic, round off error, error propagation, condition number
- Interpolation
- Numerical differentiation
- Numerical integration
- Solution of nonlinear equations
- Solution of initial value and two-point boundary value problems

Algorithms

Objectives

After completing the course, students

- i) can solve simple optimization problems using techniques such as backtracking, greedy algorithms, and dynamic programming
- ii) can analyze the running time of such algorithms and prove their correctness
- iii) can explain standard algorithms for some important problems on graphs, such as shortest paths and maximum flows, and can explain their proofs of correctness
- iv) can apply these graph algorithms to solve related problems such as bipartite matching
- v) have a basic understanding of complexity theory and can distinguish polynomial-time solvable from NP-hard problems.

Content

The efficiency of algorithms is of crucial importance in many applications. In the course Data Structures (2IL50) you have learned how to mathematically analyze efficiency, how to use design techniques like divide-and-conquer, and you have seen efficient algorithms and data structures for basic problems such as sorting and searching. In the course Algorithms we will take this one step further, by studying more advanced design techniques and algorithms, and by studying other aspects of efficiency. Many of the topics that are covered concern optimization problems, where one wants to find not just some solution to a problem but an optimal solution. The course consists of three parts. In the first part we study three general techniques for solving optimization problems: backtracking, dynamic programming, and greedy algorithms. The second part of the course deals with algorithms for optimization problems on graphs: computing shortest paths, computing maximum flows, and finding so-called matchings. In the third part of the course we study computational complexity theory by looking at NP-completeness, which investigates the limits of what is efficiently computable.

Statistics and Machine Learning

Objectives

The main focus of this course is on the theoretical and mathematical foundations of Data Mining and Machine Learning. A secondary focus is on low-level practical aspects (e.g. vanilla implementations of various models and algorithms). After the course the students will be able to:

- distinguish the two main Machine Learning paradigms of supervised and unsupervised learning.
- assess the suitability of common machine learning methods for a given learning problem
- compare weaknesses and strengths of common machine learning methods based on theoretical properties and practical considerations
- implement, and evaluate some of the most widely used methods
- reflect upon the choice of model, its theoretical properties and resulting influences on the result.

Content

Week 1: *Introduction to data mining, machine learning and optimization; Instructions: Python for Data Mining and Machine Learning; Recap mathematical basics*

Week 2: *Regression* (linear and non-linear), regularization, bias-variance trade-off, Ridge regression, Lasso

Week 3: *Classification*: Naive Bayes, K-Nearest Neighbors, Support vector machines and the Kernel Trick, Decision Trees and Random Forests; Evaluation of classifiers

Week 4: *Neural Networks* (Multi-layer Perceptrons and Convolutional Neural Networks), Backpropagation of Error, Stochastic Gradient Descent, training techniques

Week 5: *Unsupervised Learning 1*, PCA, SVD, Recommender systems, K-Means

Week 6: *Unsupervised Learning 2*, Non-convex clustering

Week 7: Guest lectures

Week 8: Preparation for Exam, example questions, Q&A

Security

Objectives

The overall aim of the course is to provide students with the basics of network and systems security, as well as traditional mitigation strategies adopted to enhance the security of IT networks.

More in detail, the course provides the following learning objectives:

1. After the course, the student can identify the security services and objectives provided by various network security protocols discussed in the course, explain their multiple configurations based on the specific security requirements of the network, and explain the security guarantees provided by such protocols with reference to specific threats and attacker models.
2. After the course, the student can identify and explain vulnerabilities affecting specific configurations of computer systems, and explain countermeasures that can be deployed to mitigate such vulnerabilities.
3. After the course, the student can describe and apply specific tools to provide security services in a general-purpose IT system.

Content

The course is divided into two main parts: frontal lectures and practical laboratories.

In the frontal lectures, the course will cover the following topics:

- Key Notions and Terms; Security Attributes/Objectives/Services, Security Protocols, and Attacks & Attacker Models.
- Cryptographic primitives; including symmetric encryption, asymmetric encryption, and Cryptographic Data Integrity Algorithms such as Hash functions, Message Authentication Codes, and Digital Signatures.
- Authentication, Authorization and Trust; User Authentication, Access Control basics, and Cryptographic Key Management & Distribution
- Network and Internet Security; SSL/TLS, SSH, Wireless Network Security, IP Security, and Network Endpoint Security (Firewall, IDS)

- Attacks and Attacker Models; Attacks on IT systems with techniques such as Buffer Overflow and SQL Injection and Web app security

The practical laboratories will cover the following topics:

- Key Management and Public-Key Certificates
- Virtual Private Networks (VPN)
- Firewalls

Multidisciplinary CBL Artificial Tutors

Objectives

Personal & professional development:

- understand, evaluate, and execute an open design process or an open scientific inquiry,
- acquire, apply, and integrate (new) disciplinary knowledge and skills (self-directed, student-controlled),
- collaborate effectively in a multidisciplinary team,
- provide and receive constructive feedback through peer- and self-assessment,
- act responsibly in every stage of the project, considering societal and environmental impact

Disciplinary knowledge and skills:

- understand and apply project-specific disciplinary knowledge and skills.

Software Development

Objectives

After finishing the Capstone Software Development course, a student is

- able to comprehend—in terms of functionality, behavior, design, development process, way of working, culture—existing (open-source) software projects.
- able to identify, analyze, and (provide suggestions to) solve issues in such software projects and their development process.

- able to improve software projects based on analyses and with respect to various quality perspectives (e.g. design and architecture, functional correctness, performance, collaborative practices, documentation) in a well-documented and systematic manner.
- able to reflect on their own practices and draw insights for improvement

In doing so, they build on what they have learned previously in the software development learning line.

Content

- In the first two weeks, a kick-off in which the goals and structure of the course are explained, groups are created, and the curated list of open-source projects is introduced; and
- Three content lectures covering basic methods to assess the software and processes of open-source projects
- In weeks 3+4, groups apply methods they learned to an open-source project of their choice from the curated list and create a mid-term report.
- In weeks 5+6, three content lectures covering advanced methods to assess the software and processes of open-source projects. At this point the students will start on their reflection essay.
- From the middle of week 6, two weeks where groups apply methods they learned to the open-source project they chose before, create a final report and presentation, and finalize their reflection essay.
- Final presentation session

Algorithmic Aspects of Game Theory

Objectives

What are the learning objectives of the course?

After successfully completing this course students will

- be familiar with the abstract definitions of non-cooperative games and strategies,

- some specific kinds of games, and how they can be used to model real-world
- situations,
- understand the notion of Nash and other types of equilibria and what they mean in
- settings modeled by games,
- know how to argue about the existence of equilibria, the complexity deciding their
- existence, and the complexity of finding them,
- understand the adaptation of some of the above to cooperative games,
- have gotten a light introduction to mechanism design including classic design
- objectives, impossibility results and approaches how to resolve them

Content

We plan for the following content:

- An introduction to fundamental game theoretic concepts including the abstract definition of games, specific games with examples, strategies and equilibria with a focus on Nash equilibria.
- Algorithmic and complexity theoretic investigation of problems relating to finding equilibria (Mini-max theorem, Nash's theorem, Sperner's lemma, Brouwer's fixed point theorem, a light discussion on the complexity classes PPAD and PLS in this context)
- A light exploration of collaborative games, specifically coalition formation games with basic concepts stability/rationality concepts studied in this context and a more detailed analysis of stable matching and the Gale-Shapely algorithm.

Additionally, we plan on discussing mechanism design theory in the context of one specific type of games which we will still determine but will be one of routing/congestion games, tournaments or auctions.